TriageLedger | Owner handoff

Revised 20 October 2026 | macOS terminal setup | 1 of 3: Prepare

Local Python CLI for explainable vulnerability triage. It combines findings, asset context and dated threat evidence to recommend priorities. Analysts retain responsibility for investigation and remediation.

Prototype only: synthetic data and illustrative policy; not a deployed service, an IEX product or a validated enterprise risk model.

1 Find and enter the repository

Open Terminal. Run each block separately. Do not paste the shell prompt (such as % or $). First try:

```
cd ~/TriageLedger
```

If that folder is missing, locate your existing checkout:

```
find ~ -type d -name TriageLedger -prune 2>/dev/null
```

Enter the returned folder using cd followed by its quoted full path. If no checkout exists, clone a new copy:

```
cd ~
git clone https://github.com/RBdishoo/TriageLedger.git
cd TriageLedger
```

Clone only if you do not already have a checkout. If Git is unavailable, complete the command-line tools installation before retrying.

2 Confirm the project root

```
pwd
ls pyproject.toml
```

Continue only when pwd ends in TriageLedger and ls displays pyproject.toml. The earlier errors occurred because installation ran in your home folder instead.

3 Activate a project-local environment

If a previous .venv is active, run deactivate first. Leave the accidental home-folder environment in place; deleting it is unnecessary. Then run:

```
python3.12 --version
python3.12 -m venv .venv
source .venv/bin/activate
python -c "import sys; print(sys.executable)"
```

Expected: Python 3.12.x and an executable path ending in TriageLedger/.venv/bin/python. If python3.12 is not found, stop and install Python 3.12 before continuing.

Important: stop at the first failed command. Continue on the next page only after the directory and interpreter checks succeed.

Install, check, and run

TriageLedger | 2 of 3: Execute

4 Install development dependencies

```
python -m pip install -e ".[dev]"
```

Wait for successful completion. If installation fails, do not run the tests or demo; resolve the installation error first.

5 Run quality checks

```
python -m pytest
python -m ruff check .
python -m ruff format --check .
```

Run each command in order. Stop if a check fails and retain its full output. Do not change expected test results merely to make a failure disappear.

6 Evaluate the demonstration data

The output directory must not already exist. Use the name below only if it is unused; otherwise choose a new name and use it consistently.

```
python -m triageledger.cli evaluate \
  --assets samples/assets.csv \
  --findings samples/findings.csv \
  --threat-evidence samples/threat_evidence.csv \
  --policy policies/demo-policy-v1.json \
  --as-of 2026-10-19T00:00:00Z \
  --output-dir artifacts/demo_run_02
echo $?
```

Keep each backslash as the final character on its line, with no trailing spaces. If PDF copying introduces unusual whitespace, type the command manually.

Exit	Interpretation
0	Completed with no review flags.
1	Operational, publication or unexpected internal failure.
2	Invalid arguments, inputs or policy.
3	Completed with review flags; expected for the supplied demo.

7 Confirm the outputs

```
ls artifacts/demo_run_02
```

Expected six files: decisions.json, queue.csv, review.csv, briefing.md, diagnostics.json and manifest.json. Do not generate into samples/demo/; it contains committed reference artifacts.

Quick troubleshooting

No pyproject.toml: wrong folder; return to steps 1-2.
No pytest, ruff or triageledger: installation failed or the wrong environment is active; repeat the interpreter check and step 4.
Destination exists: use a fresh output name; keep prior results.
Pip upgrade notice: informational; it was not the cause of the reported installation failure.

Demonstrate and maintain

TriageLedger | 3 of 3: Owner reference

Three-minute demonstration

1. **Compare:** docs/comparison.md shows the same 15 actionable findings reordered from a severity-only baseline. One not_affected finding is excluded from both comparisons.

2. **Explain:** artifacts/demo_run_02/decisions.json links recommendations to matching rules, evidence and uncertainty flags.

3. **Summarize:** artifacts/demo_run_02/briefing.md shows priorities, review needs and limitations. A recommendation supports an analyst; it does not replace judgment.

Three examples to know

Finding	Change	Reason
F-001 | high	4th → 1st; P1	P-01: confirmed applicability plus fresh active exploitation. Conflicting not_observed evidence remains flagged.
F-004 | critical	2nd → 7th; P2	Nine-day-old evidence exceeds the seven-day threshold. Effective exploitation is unknown; P-03 assigns P2. Six P1 findings precede it.
F-011 | unknown	15th → 8th; P2	P-06 expedites clarification of unknown severity. It does not establish high risk.

Positions compare the severity-only and context-aware queues. A changed position does not prove improved security outcomes.

Engineering essentials

Input consistency: cli.py captures bounded snapshots for parsing, policy loading and hashing.
Evidence: policy.py preserves stale active evidence as flagged history; fresh active evidence takes precedence.
Integrity: reporting.py writes hashes. Comparison with a trusted, unchanged manifest can detect changes; it does not authenticate the manifest or establish input truth.
Reproduction: requires identical input bytes, policy, evaluation time, tool version, input-path strings and consistent serialization conditions.

Controlled maintenance

1. Document the intended behavior; increment the policy version for policy changes.
2. Add independently specified regression tests and make the smallest justified change.
3. Run focused tests, the full suite, lint and format checks.
4. Generate into a fresh directory; inspect differences before replacing references.
5. Update affected documentation, commit reviewed changes and inspect CI.

Boundaries to state clearly

Synthetic data and uncalibrated thresholds; ordinal recommendations, not risk probabilities. No SaaS API, multi-user access, live scanner integration or automatic remediation. Staged publication does not guarantee crash durability or concurrent-writer safety. CSV mitigation is not a universal spreadsheet-safety guarantee.

Source: repository README and the prior owner handoff. Repository reference: 0c62186. No tests were rerun to prepare this document.

https://github.com/RBdishoo/TriageLedger